

Assignment 1 — Safe Generative AI

Reward Modelling, PPO and DPO Fine-Tuning

Sharvari Wanjari
Roll No: 22CS30049

February 2026

HuggingFace Profile: <https://huggingface.co/mysteriousgirl>

1 Overview

This assignment involved building an RLHF-style pipeline to align GPT-2 Medium to prefer safer, more human-aligned responses. The work is split into three parts:

- **Task A (Reward Model):** Train a BERT-based reward model on pairwise preference data.
- **Task A (PPO):** Use the reward model to fine-tune GPT-2 Medium via PPO across different PEFT strategies.
- **Task B (DPO):** Fine-tune GPT-2 Medium directly on preference pairs using Direct Preference Optimisation, no reward model needed.

All trained checkpoints are on HuggingFace and linked in Section 2.

2 HuggingFace Model Links

Checkpoint	Link
Reward Model (BERT)	https://huggingface.co/mysteriousgirl/reward-model-bert
PPO – Full	https://huggingface.co/mysteriousgirl/ppo-gpt2-full
PPO – LoRA	https://huggingface.co/mysteriousgirl/ppo-gpt2-lora
PPO – QLoRA	https://huggingface.co/mysteriousgirl/ppo-gpt2-qlora
DPO – Full	https://huggingface.co/mysteriousgirl/dpo-gpt2-full
DPO – Prefix	https://huggingface.co/mysteriousgirl/dpo-gpt2-prefix
DPO – LoRA	https://huggingface.co/mysteriousgirl/dpo-gpt2-lora
DPO – QLoRA	https://huggingface.co/mysteriousgirl/dpo-gpt2-qlora

3 Task A — Reward Model

3.1 Architecture and Training

I used `bert-base-uncased` as the backbone with a single linear layer on top of the [CLS] token to produce a scalar reward. The input is the concatenation of the question and the response.

$$r(x, y) = W \cdot \text{BERT}_{\text{CLS}}([x; y]) + b$$

Training used the Bradley-Terry pairwise loss:

$$\mathcal{L} = -\log \sigma(r_{\text{chosen}} - r_{\text{rejected}})$$

Hyperparameter	Value
Base model	bert-base-uncased
Optimizer	AdamW, lr = 2e-5
Scheduler	Linear warmup (50 steps)
Epochs	3
Batch size	16
Max seq length	256 tokens
Training samples	5,000 (random_state=42)

The model trained for 3 epochs and loss decreased steadily each epoch. After training, the full state dict (`full_reward_model.pt`) and backbone were saved and uploaded to HuggingFace.

4 Task A — PPO Fine-Tuning

4.1 Setup

The reward model was loaded from HuggingFace and used as the scoring function. GPT-2 Medium was the policy model, fine-tuned using TRL’s PPOTrainer.

Parameter	Value
Base LM	gpt2-medium
PPO epochs	3
Batch size	8
Mini-batch size	4
Learning rate	1.41e-5
Max prompt length	128 tokens
Max new tokens	64
Prompts per epoch	400

4.2 Strategies

Full fine-tuning and LoRA both ran successfully. **QLoRA** ran but after fixing errors. **Prefix Tuning was skipped** — TRL 0.8.6’s PPOTrainer does an `isinstance` check on the policy model that does not recognise PEFT-wrapped prefix models. Multiple workarounds were attempted (manually registering the class in `SUPPORTED_ARCHITECTURES`, building a custom `PreTrainedModelWrapper` subclass) but all resulted in sequence length mismatch errors that couldn’t be resolved without downgrading transformers below the minimum required version.

4.3 Reward per Epoch

Strategy	Epoch 1	Epoch 2	Epoch 3	Note
Full	-4.1150	-3.9888	-3.0348	Steadily improving
LoRA	-4.4377	-2.8928	-2.7283	Converges faster
QLoRA	—4.5662	—4.7001	—4.6282	Converges slower
Prefix	N/A	N/A	N/A	Skipped (TRL conflict)

Rewards are negative log-probs normalised by sequence length, so values closer to zero are better. Both strategies show clear improvement over 3 epochs, with LoRA showing a bigger jump in epoch 2.

5 Task B — DPO Fine-Tuning

5.1 Setup

DPO doesn’t need a reward model — the policy is trained directly on (prompt, chosen, rejected) triples. The reference model is a frozen copy of GPT-2 Medium. The DPO loss is:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right] \right)$$

where $\beta = 0.1$ controls how much the policy is allowed to deviate from the reference.

Parameter	Value
Base LM	gpt2-medium
Epochs	3
Batch size	8
Max seq length	96 tokens
Learning rate	2e-5
Beta	0.1
Scheduler	Cosine, 10 warmup steps
Grad clip	1.0
Training data	5,000 samples

5.2 Note on Missing Logs

The Full and Prefix DPO runs were done in earlier Kaggle sessions. The checkpoints were uploaded to HuggingFace before the session ended, but the Kaggle runtime reset cleared the cell outputs. So those cells don't show training logs in the submitted notebook. LoRA and QLoRA runs were done in the final session and do have visible outputs.

All four strategies (Full, Prefix, LoRA, QLoRA) were trained and the checkpoints are on HuggingFace.

6 Evaluation

6.1 Setup

Models were evaluated on 6,000 samples from `preference_test.csv`. For each prompt, the model generates a response which is scored against the ground-truth preferred response using:

- **BLEU** — n-gram overlap
- **ROUGE-L** — longest common subsequence recall
- **Semantic Similarity** — cosine similarity using `sentence-transformers/all-MiniLM-L6-v2`

The baseline scores the more-preferred response against the less-preferred response directly (i.e., how different human-written responses are from each other).

6.2 Results

Strategy	BLEU	ROUGE-L	Sem-Sim
Baseline (reference)	0.0259	0.1681	0.6250
PPO – Full	0.0000	0.0020	0.0363
PPO – LoRA	0.0000	0.0022	0.0344
PPO – QLoRA	0.0008	0.0603	0.2994
DPO – Full	0.0021	0.0764	0.4391
DPO – LoRA	0.0014	0.0863	0.2644
DPO – QLoRA	0.0014	0.0825	0.2808

Table 1: Evaluation results. Best model highlighted in bold.

6.3 Analysis

DPO consistently outperforms PPO. DPO-Full has the highest semantic similarity (0.4391) across all models, and DPO-QLoRA has the best ROUGE-L (0.0921). PPO-Full and PPO-LoRA both have near-zero scores, which suggests the PPO training destabilised the model's output distribution significantly — likely because the reward model signal was noisy and the policy overfit to gaming the reward rather than producing coherent responses.

PPO-QLoRA is an interesting exception. It scores much better than PPO-Full or LoRA (Sem-Sim 0.2994). QLoRA's quantisation likely acts as implicit regularisation,

preventing the policy from drifting too far during RL training and keeping outputs more coherent.

Why all models are below baseline. The baseline metric (0.625 Sem-Sim) measures similarity between two human-written responses to the same question — which is naturally high because humans tend to write similar things. A GPT-2 Medium model trained for 3 epochs on 5,000 samples can’t match that. The models are learning a preference direction, not trying to reproduce human text verbatim.

Prefix tuning for DPO worked fine but faced issues in evaluation, unlike in PPO where it had to be skipped entirely.

7 Reproducibility

Item	Details
Platforms	Google Colab (reward model, PPO), Kaggle GPU (DPO)
GPU	NVIDIA T4
Dataset	preference_train.csv + preference_test.csv (course dataset)
Seed	<code>random_state=42</code> for all sampling
Train size	5,000 samples
transformers	4.40.0
trl	0.8.6
peft	0.10.0
accelerate	latest compatible
bitsandbytes	latest compatible

To reproduce: use the submitted notebooks with the library versions above, point CSV paths to the preference dataset, and replace the HF token/username with your own.

Submitted Files

File	What it does
<code>Task_A_reward.ipynb</code>	Trains the BERT-based reward model on pairwise preference data using Bradley-Terry loss, saves weights and uploads to HuggingFace.
<code>Task_A_alignment.ipynb</code>	Loads the reward model from HuggingFace, then fine-tunes GPT-2 Medium via PPO using Full, LoRA, and QLoRA strategies. Prefix tuning is documented but skipped due to TRL version incompatibility.
<code>Task_B.ipynb</code>	Fine-tunes GPT-2 Medium using DPO (no reward model needed) across all four strategies: Full, Prefix, LoRA, and QLoRA. Each trained checkpoint is uploaded to HuggingFace after training.
<code>Task_C.ipynb</code>	Code for evaluation of HF endpoints.
<code>Report.pdf</code>	This report.

8 Conclusion

DPO turned out to be more practical and effective than PPO for this task. It was simpler to train, didn't need a separate reward model inference step during training, and produced better outputs by every metric. Among PEFT strategies, Full fine-tuning gave the best quality for DPO, while LoRA was close behind with far fewer trainable parameters. QLoRA is the most memory-efficient option and still produces reasonable results, particularly in the PPO setting where its regularisation effect seems to help.

The main difficulty was Prefix Tuning with PPO — the TRL version required by the assignment has compatibility issues with PEFT's prefix implementation that I couldn't work around. For DPO it was fine.